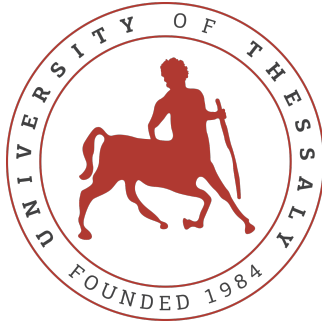




UNIVERSITY OF THESSALY
POLYTECHNIC SCHOOL
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Speech Processing Algorithms

by

Petroglou Spyridon 03185

A report submitted to the University of Thessaly
in partial fulfillment of the requirements for the degree of
B.Sc., (Hons) in Electrical and Computer Engineering.

Volos, 2024

Contents

1	Introduction	2
2	Part A: Extract Audio Features and Apply them to Basic Speech Processing Algorithms	3
2.1	Create Spectrograms	8
2.2	Discriminate Voiced from Unvoiced Speech Segments	10
2.2.1	Noise Modelling	10
2.2.2	Create and Apply the Algorithm that detects Voiced and Unvoiced Regions	11
2.3	Pitch Detection using the Autocorrelation Technique with Center Clipping . . .	14
2.3.1	Speech/Background Sound Discrimination	14
2.4	Linear Predictive Analysis of Two Speech Segments	17
3	Part B: A Simple Vowel Synthesizer	22
3.1	Human Vocal System	22
3.2	Synthesize Vowels	28
3.3	Create the Necessary Spectrograms	32

1 Introduction

In this report, I will present my assignment for the course *Speech Processing Algorithms*. The assignment is divided into two parts. In the first part, I will implement some basic algorithms on a simple speech signal, while in the second part, I will create a vowel synthesizer and use it to synthesize six vowels. The algorithms in the first part offer high accuracy and have been widely used in speech processing, especially before the Neural Network era. The vowel synthesizer in the second part aims to mimic the human vocal system, which will be described in detail later, and produce vowels digitally. For simplicity, the synthesizer will operate at a constant fundamental frequency (125 Hz), whereas in human speech, this frequency varies between 50 Hz and 400 Hz. Despite this limitation, the synthesizer produces remarkable results, allowing the vowels to be easily distinguished. The project has been fully developed in Python, specifically using Jupyter Notebook, allowing the user to view and interpret the results easily at each step.

2 Part A: Extract Audio Features and Apply them to Basic Speech Processing Algorithms

In this part of the assignment, I will use some fundamental concepts in the field of speech processing, such as the Short-Time Energy (or Short-Time log Energy in Logarithmic scale), the Short-Time Zero Crossing Rate, the Linear Predictive Analysis Coefficients (or LPC shortly) and the Short-Time Autocorrelation, to create some basic speech processing algorithms, such as voiced/unvoiced discrimination and pitch detection. The experimental signal that I will use during my analysis will be my full name, which is located in the 'wav_files' folder, recorded with a sampling rate of 16kHz. For the sake of simplicity, I defined the following constants:

- **FRAME_LENGTH**: The length of each frame that will be used for almost all Short-Time analyses (apart from Pitch Detection). It is set at 160 samples.
- **HOP_LENGTH**: The number of samples that the frame is moving in each measurement. The default value is set at 40 samples, which means that the frames are having 25% overlap.
- **PITCH_FRAME_LENGTH**: The length of each frame that will be used for Pitch Detection. It is set at 480 samples.
- **PITCH_HOP_LENGTH**: The hop length for Pitch Detection. Its default value is 180 samples (20msec overlap)
- **CUTOFF_HP**: The Cutoff Frequency of the High-Pass Filter. It is set at 200Hz.
- **KSIZE**: The size of the kernel of the median filter (in other words, how many samples I will use to find the median value). The larger the value of KSIZE, the smoother the curve.

The default value is set at 17.

And I created these functions too:

- **create_spectrogram**: It is used to create the spectrogram by applying the STFT in overlapping frames. It takes as input the analysis signal, the size N of FFT, the constants FRAME_LENGTH and HOP_LENGTH and the window type. It returns the amplitude of each STFT (computed with both absolute value function and square function). The amplitude is in a matrix form, where each column represents a time bin (a frame) and each row represents a frequency bin. Each index value contains the frequency amplitude in a specific frame (in decibels).
- **plot_spectrogram**: It takes as input the returned arrays from the 'create_spectrogram' function (and a title), and plots the spectrograms.
- **linear_filter**: This function creates and applies linear filters using butterworth filters. It takes as input the signal that is going to be filtered, the cutoff frequency, the sampling rate, the type of the filter (if it is lowpass or highpass), and the order of the filter. It returns the filtered signal. For this project, I have used only highpass filter.
- **median_filter**: This function uses the medfilt() function from scipy.signal library to perform non-linear smoothing. The function takes as input the rough energy and zero crossing signals, along with the kernel size (the constant KSIZE), applies the median filter in both signals, and returns the smoothed versions of them.
- **short_time_log_energy_func**: Computes the Short-Time Energy of the input signal using the sliding window technique, with frame characteristics defined at the constants. It returns the Short-Time Energy both in linear and log scale. The energy in log scale is normalized using the following formula:

$$\log(E_n) = \log(E_n) - \max(\log(E_n))$$

and the maximum value will occur at 0 dB.

- **short_time_zero_crossings_func:** Computes the Short-Time Zero Crossing Rate of the input signal using the sliding window technique, with frame characteristics defined at the constants. It returns a vector that contains the number of Zero Crossings in each frame.
- **voiced_unvoiced_frame_classification:** This function is used to classify frames as 'Voiced' or 'Unvoiced'. For a frame to be 'Unvoiced', it should have more Zero Crossings than a specific threshold named IZCT. For a frame to be 'Unvoiced', it should have a larger Energy value than a specific threshold named ITR. These thresholds have arose after taking some statistical measurements in the first 100 msec of the signal, and they are also given as inputs to the function (details about these constants later). Furthermore, I will give higher priority to unvoiced frames, which mean that if a frame has been classified in both classes, it will be removed from the voiced class. Obviously, the function returns two vectors: one containing the voiced frames and the other containing the unvoiced frames.
- **classify_samples:** This is used after the 'voiced_unvoiced_frame_classification' function, and for each frame, it classifies all of its samples as 'Voiced' or 'Unvoiced'. It takes as input either the vector that contains the voiced frames or the vector that contains the unvoiced frames along with the frame characteristics, and returns the samples that belong to the corresponding class (in a vector). For each frame, it finds the 160 samples that each frame contain by using these formulas:

$$Begin = Rx[i - 1]$$

$$End = Rx[i - 1] + L$$

where R is the hop length, L is the frame length and $x[i - 1]$ is the (i-1)-th frame. Because the frames are overlapping, I need to find a method not to include duplicate values. That's why I defined three constants named *const1*, *const2* and *const3* and define from which index the samples will start to append into the vector in case of overlapping frames. For example, the 26th frame contains the samples 1000-1160 and the 27th frame contains the samples 1040-1200. In this case, *const1* helps me to discard the values 1040-1160 when I

insert them into the vector, in order to avoid duplicate values that lead to memory waste. If I had the 26th and the 28th frame, then I would have used the *const2* to discard the values 1080-1160. Generally, there are 3 cases of overlap; when I have 2 consecutive frames, when I have 2 frames that they are far by 2, and when I have 2 frames that they are far by 3. If two frames are far by 4 or more, there is no overlap (these assumptions are true only at 75% overlap). In case of non-overlapping frames, just append all values to the vector.

- **speech_silence_discrimination_func:** The algorithm that is used to discriminate speech from background noise is located here. It takes as input the Short-Time log Energy and the Short-Time Zero Crossing Rate that arose from the speech signal, and three constants named ITU, ITR and IZCT. Essentially, this algorithm places the beginning frame in the first frame that exceeds ITR, and after that, it exceeds ITU before falling down to ITR, and the end frame, in the last frame that exceed ITR, and then ITU, before falling down to ITR again. Then, these frames may be adjusted by using the zero crossings. If there are more than 4 zero crossings greater than IZCT threshold in the first 25 samples outside the interval, the frames are moving to the last frame (the farthest one) that just exceeds the IZCT. The function returns the beginning and the end frame that are found in each step of the algorithm.
- **center_clipping_func:** This function performs a 3-level center clipping to the input signal. Apart from the input signal, it takes as input the frame characteristics too. Firstly, the function performs framing in the input signal and creates a 2D array, in which each column contains a frame. Then it slices this array horizontally to 3 equal parts. In a 30msec input signal (480 samples in 16kHz sampling rate), each part represents a 10msec interval. To set the clipping level, it finds the maximum absolute value of the first and third intervals, and then, it is set at a fixed percentage of the minimum of these values (in my case, it is set at 80%). Then, it performs the 3 level clipping to the input signal, where a value is set at 1, if a sample exceeds the clipping level, at -1 if the sample exceeds the negative value of the clipping level, and zero otherwise. The function returns the clipped signal (with values only at -1, 0 and 1), along with the autocorrelation of the clipped signal, which is

performed in each frame of the framed signal (in each column of the 2D frame array).

- **create_autocorrelation_matrices:** The purpose of this function is to create a $p \times p$ autocorrelation matrix R , and a target vector r , which will be used for LPC analysis. It takes as input a voiced and an unvoiced segment along with the order p of the LPC analysis, and it returns the matrix R and the target vector r , for both voiced and unvoiced segments. To construct the R matrix, it performs the autocorrelation of both segments, from lag 0 up to lag $p+1$. The R matrix is a $p \times p$ Toeplitz Matrix of the form

$$\begin{bmatrix} R[0] & R[1] & R[2] & \dots & R[p-1] \\ R[1] & R[0] & R[1] & \dots & R[p-2] \\ R[2] & R[1] & R[0] & \dots & R[p-3] \\ \dots & \dots & \dots & \dots & \dots \\ R[p-1] & R[p-2] & R[p-3] & \dots & R[0] \end{bmatrix}$$

while r is a vector, that contains

$$\begin{bmatrix} R[1] & R[2] & R[3] & \dots & R[p] \end{bmatrix}$$

This matrix and this vector will be used to solve a linear system $Ra = r$, where a is a vector that contains the LPC coefficients. The function returns the autocorrelation array and the target vector for both voiced and unvoiced segments.

- **compute_mse:** Obviously, this function is used for computing the Mean Squared Error of a signal, by evaluating this function

$$E_n = R_n[0] - \sum_{k=1}^p a_k R_n[k]$$

where $R_n[0]$ is the energy of the signal, a_k are the LPC coefficients, $R_n[k]$ is the target vector and p the order of LPC analysis. All these values are given as inputs in the function.

- **all_pole_transfer_function:** It creates the all-pole filter $H(z)$, which is equal to

$$H(z) = \frac{G}{1 - \sum_{k=1}^p a_k z^{-k}}$$

where G is the gain and a_k are the LPC coefficients (given as inputs to the function). The function computes the frequency response of the all-pole filter $H(z)$ and returns the magnitude of it, along with an array w of normalized frequencies (useful for the plot).

2.1 Create Spectrograms

After the recording of my full name, the first thing that I have to do is to create some spectrograms. Spectrograms are really useful in Audio Processing because they show us how frequencies are evolving over time, and also, they can be used as inputs in Convolutional Neural Networks that make Audio Classification, Speech Recognition, Music Genre Classification etc. They are created by applying the Short-Time Fourier Transform (STFT) in small overlapping windows. By this way, we have information for both time and frequency domain. To create the spectrograms, I will use the custom-made function 'create_spectrogram' that is described in the first bullet above, and to plot them, I will use the other custom-made function, named 'plot_spectrogram'.

For the purpose of my assignment, I will create 2 spectrograms, using a Hamming window and 5msec (80 samples) hop length, but in the first case, the frame length will be 10msec (160 samples or 50% overlap), and in the second case, the frame length will be 100msec (1600 samples or 5% overlap). The created spectrograms are shown in Figure 2.1 and 2.2

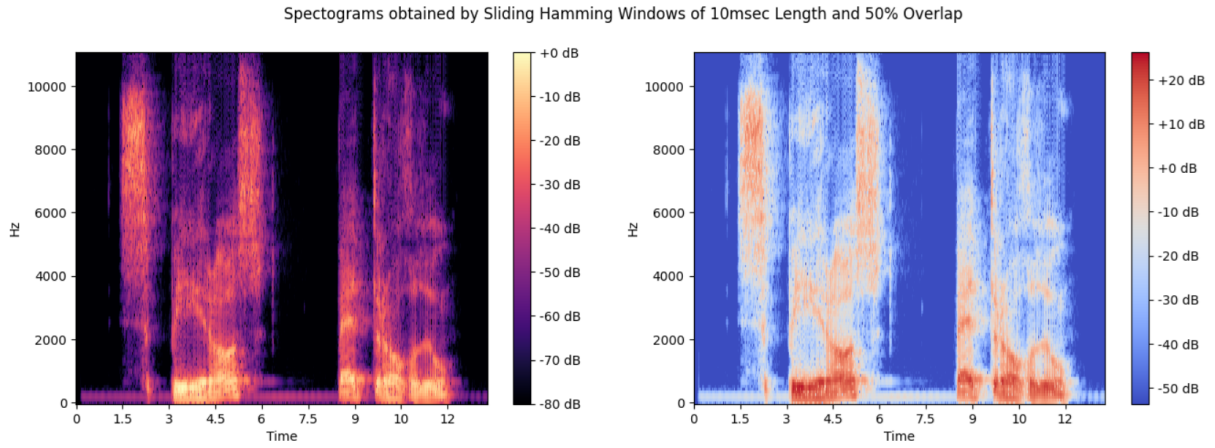


Figure 2.1: Spectrograms using 10msec window in linear scale

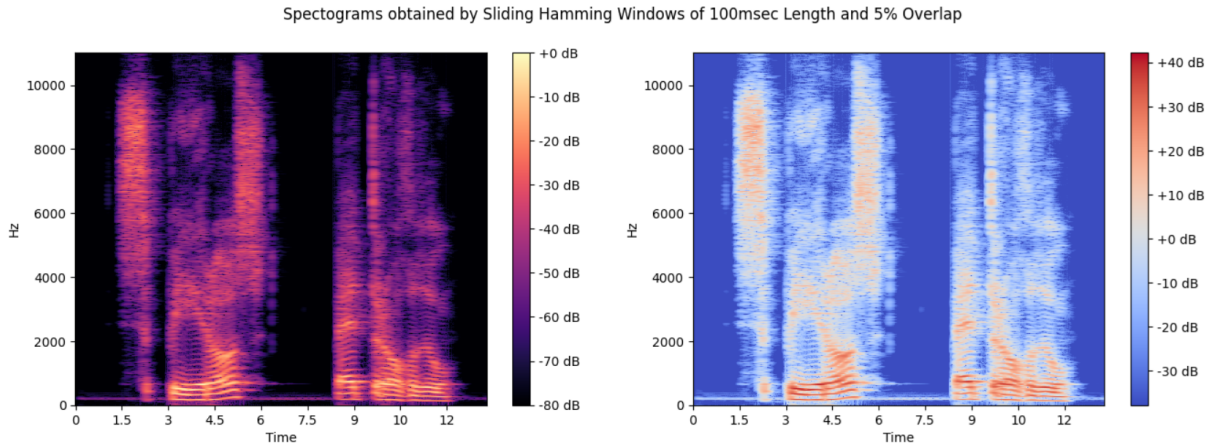


Figure 2.2: Spectrograms using 100msec window in linear scale

We can clearly see the effect of the window length in our analysis. The first spectrogram (the black one) is obtained using the absolute value of the amplitude of the signal, whereas the second spectrogram (the blue one) is obtained by using the square of the amplitude of the signal. The upper spectrograms are the wideband ones, with a bandwidth equal to $1 / \text{frame duration} = 1 / 10\text{msec} = 100\text{Hz}$, while the bottom spectrograms are the narrowband ones, with a bandwidth equal to $1 / \text{frame duration} = 1 / 100\text{msec} = 10\text{Hz}$.

The main difference lies in the orientation of the stripes. In the wideband spectrograms, the stripes are vertical because they have a good temporal resolution, but poor frequency resolution. In this case, it is easy to capture rapid changes in the input signal, but harder to distinguish

between two closely related frequencies. On the other hand, the narrowband spectrograms have horizontal strides and provide better frequency resolution, facilitating the separation of closely spaced frequencies, but they cannot capture rapidly changing events in the input signal. In other words, as the bandwidth of the bandpass filter increases, the frequencies are classified more accurately because more frequency bins are created, improving the frequency resolution of the spectrogram, but the temporal resolution becomes poorer.

2.2 Discriminate Voiced from Unvoiced Speech Segments

In this section, I will implement a simple algorithm for a common speech processing task, which is to separate the voiced segments of the speech, from the unvoiced ones. Briefly, voiced sounds are the ones that occur when the vocal cords vibrate due to the air flow, producing quasi-periodic waveforms. Unvoiced sounds are not produced by vocal chord vibration, but by turbulences from the air that is forced through some partial constrictions in the vocal tract. Thus, the waveforms of these sounds are aperiodic.

To classify each sample as 'Voiced' or 'Unvoiced', I will use 2 Short-Time Features, the Short-Time log Energy and the Short-Time Zero Crossing Rate. I will use the log energy to classify the voiced sounds, and zero crossings to classify most of the unvoiced sounds. But first, I have to model the noise to make my algorithm less sensitive to it.

2.2.1 Noise Modelling

To obtain a quite strong model of the noise, I will use the first 100msec of my input signal (usually, this small interval contains no speech). As I noticed, the first 2.560 samples (around 150msec) contain zeros, so firstly, I will trim my input signal to discard this part. The name of my new signal will be *speech_signal_sliced*, and from this signal, I will process the first 100msec (1.600 samples), to model the noise.

For the noise modelling, I will extract 4 statistical features, named *eavg*, *esig*, *zcavg*, *zcsig*, where:

- **eavg**: Mean Value of the log Energy inside the noise interval

- **esig**: Standard Deviation of the log Energy inside the noise interval
- **zcavg**: Mean Value of the Zero Crossings inside the noise interval
- **zcsig**: Standard Deviation of the Zero Crossings inside the noise interval

These features provide a rough statistical characterization of the background noise, and I will use them to define some constants. These constants have been referred before and are named IZCT (Zero Crossing Threshold), ITR (Lower Energy Threshold) and ITU (Upper (or Strict) Energy Threshold). They are given by the following formulas:

- $IZCT = \max(35, zcavg + 3 * zcsig)$
- $ITU = \text{const between } -10 \text{ and } -20 \text{ dB}$
- $ITR = \max(ITU - 10, eavg + 3 * esig)$

2.2.2 Create and Apply the Algorithm that detects Voiced and Unvoiced Regions

After the acquisition of these constants, it's time to find the Short-Time log Energy and Short-Time Zero Crossing rate of the speech signal, by using the appropriate functions.

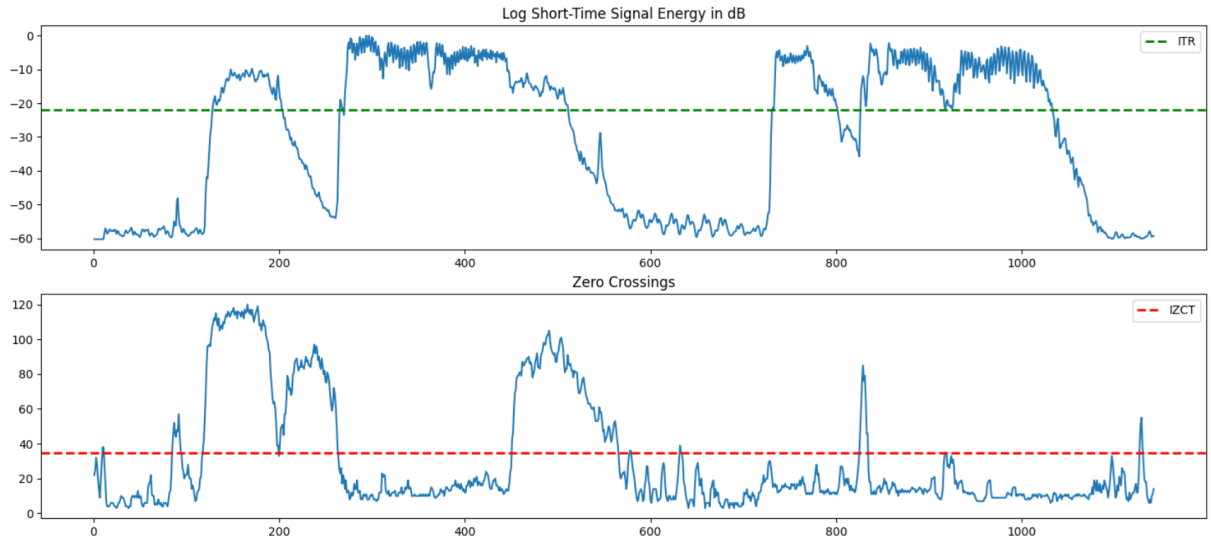


Figure 2.3: Short-Time log Energy and Short-Time Zero Crossing Rate of each frame, along with useful constants

Figure 2.3 shows us the output of these functions, along with the constants IZCT and ITR (I will use the ITU constant later). This function may need a smoothing filter for better results, but we can have a first estimate of the classification just by looking at these plots. After that, I need to classify each frame as 'Voiced' or 'Unvoiced' by calling the 'voiced_unvoiced_frame_classification'. As I have already said, the frames above the red line in Zero Crossings will be classified as 'Unvoiced', while the frames above the green line in log energy will be classified as 'Voiced' only if they have not already been classified as 'Unvoiced'.

After the frame classification as 'Voiced' and 'Unvoiced', I need to classify the samples that each frame contains too. For this purpose, I will use the custom-made function named 'classify_samples'. This function is called twice, one time to match the unvoiced segments with the corresponding samples, and one more time, to match the voiced segments. Details about the functionality of this function have already been covered above.

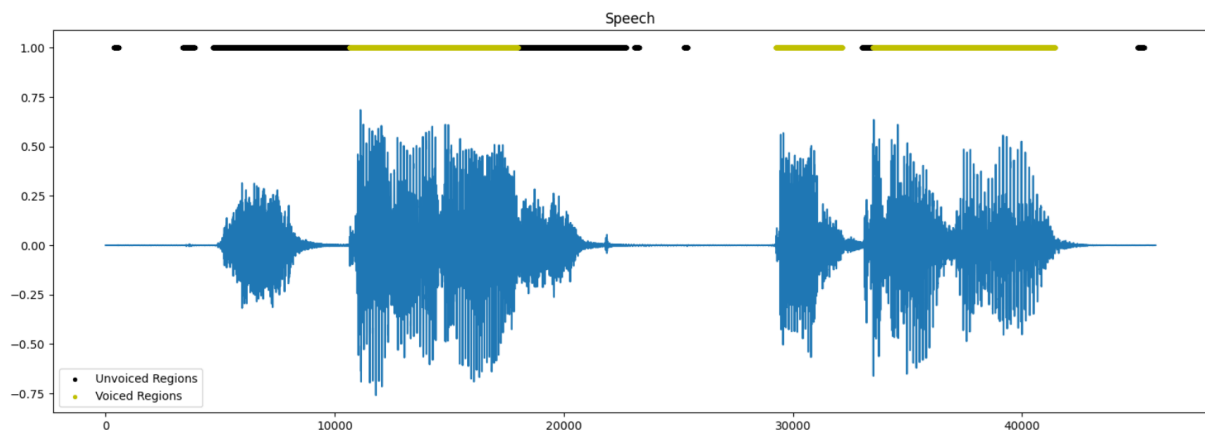


Figure 2.4: Speech Classification in Voiced and Unvoiced Parts

Figure 2.4 provides us the first plot of the speech signal along with its labels. As you can see in the legend on the bottom left, unvoiced parts are shown with a black dot, while voiced parts are shown with a yellow dot. Fortunately, the sharpness of the Short-Time features didn't affect the classification, but in some cases, there may be voids in between samples that belong to the same class. For this purpose, I have created a function that applies a smoothing filter (the median filter), to reduce this effect and to remove some outliers too. The median filter is a non linear filter that finds the median in a specific region (defined by a constant), and moves forward

to find the median of the next region, till it reaches the end of the signal.

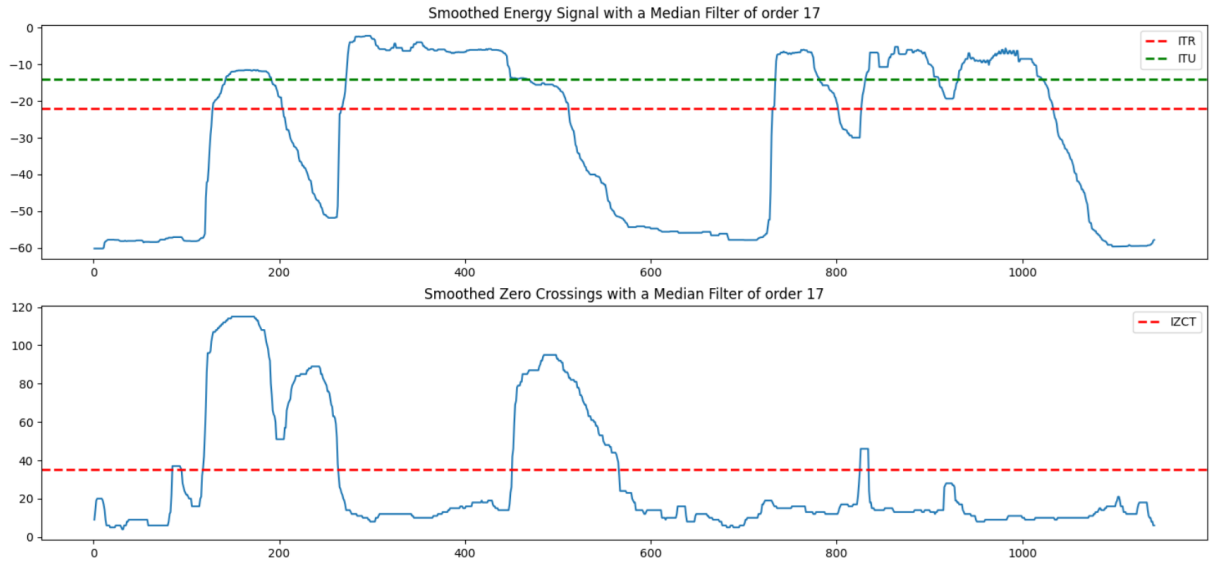


Figure 2.5: Short-Time log Energy and Short-Time Zero Crossing Rate of each frame after the application of the median filter, along with useful constants

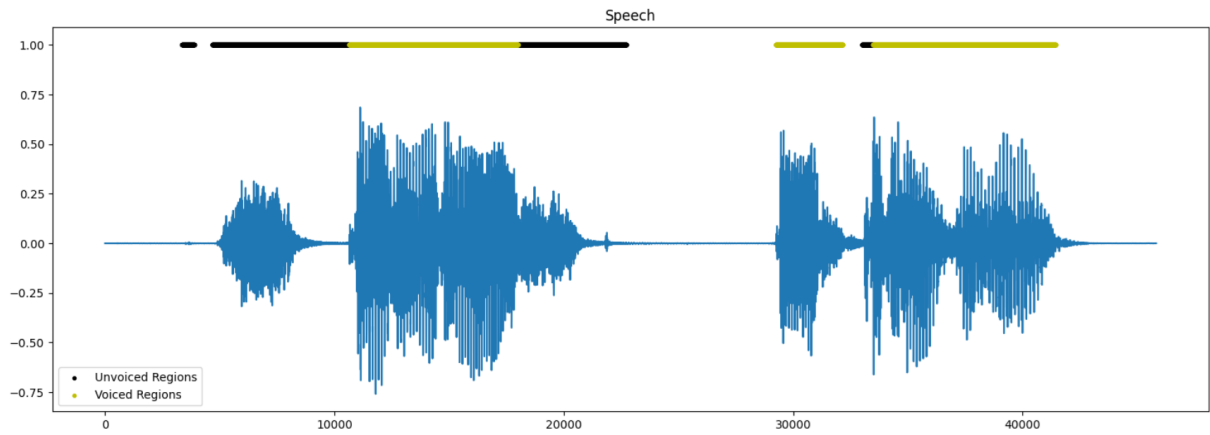


Figure 2.6: Speech Classification in Voiced and Unvoiced Parts after the Application of the Median Filter

Figure 2.5 shows us the smoothed curves of Short-Time features, while Figure 2.6 shows us the classification in voiced/unvoiced regions based on the new smoothed curves. We can notice that the outliers have been significantly removed in comparison with the previous classification.

2.3 Pitch Detection using the Autocorrelation Technique with Center Clipping

Another common, but a more challenging, task in speech processing, is to detect the pitch period of voiced samples. As I have said before, voiced sounds are quasi-periodic, which mean that after a specific time, the signal, almost, repeats itself (it's almost periodic, but not fully periodic). But using specific methods we are able to find this period that the signal, almost, repeats itself. The method that I will use for this task is the Autocorrelation Method.

2.3.1 Speech/Background Sound Discrimination

Now, for this task, I will apply an algorithm to find the beginning and the end of the speech signal. This process is described analytically in the 'speech_silence_discrimination_func', which is the function that is used for this task. The trimmed signal after the application of the algorithm is shown in Figure 2.7. Here we can also see the final result of the voiced/unvoiced classification. It seems that the algorithm has found correctly all voiced segments, most unvoiced segments (especially the unvoiced fricative /s/ twice), it detected only a little of the unvoiced stop /t/, and failed completely to detect the unvoiced stop /p/ in the beginning of the surname.

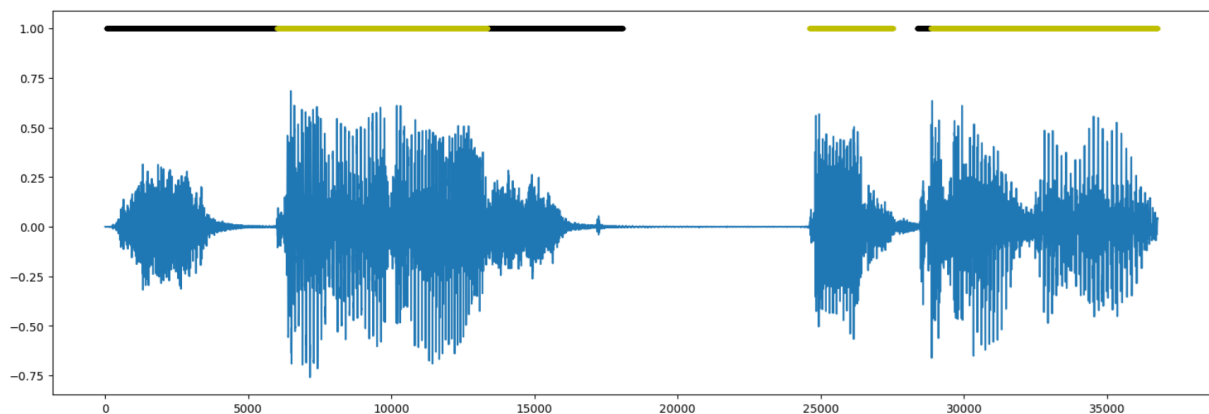


Figure 2.7: Trimmed Speech Signal along with its Classification in Voiced and Unvoiced Parts

The locations of the frames can be seen at Figure 2.8. We can see that the beginning frame was re-adjusted (suppose due to the high zero crossing rate of the unvoiced consonant /s/ in the

beginning of the speech), while the end frame didn't move during the evaluation of the zero crossing rate.

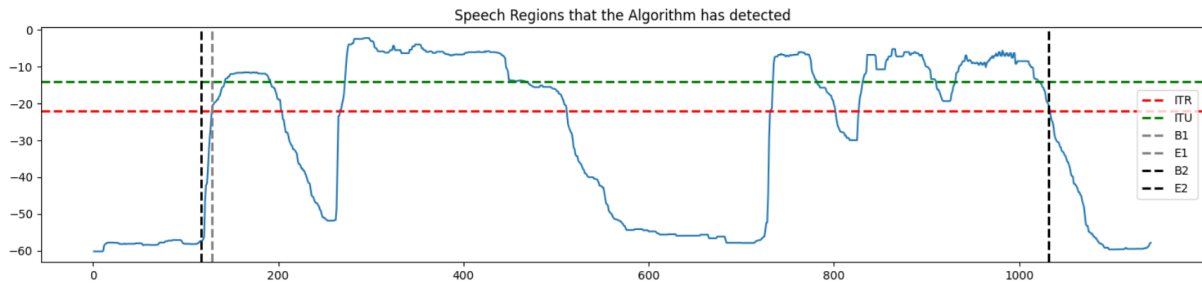


Figure 2.8: Beginning and End Frames of the Speech Discrimination Algorithm

After that, I used the custom-made function, named 'center_clipping_func', to create the 3-level center clipping and calculate the autocorrelation of each frame.

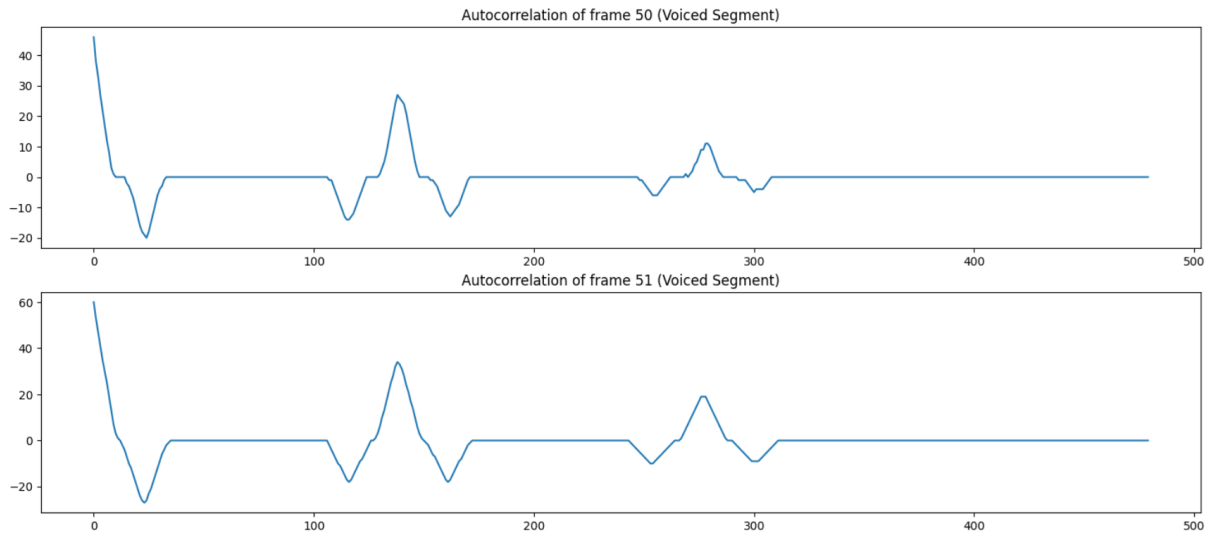


Figure 2.9: Autocorrelation Function applied at 2 Fully Voiced Frames

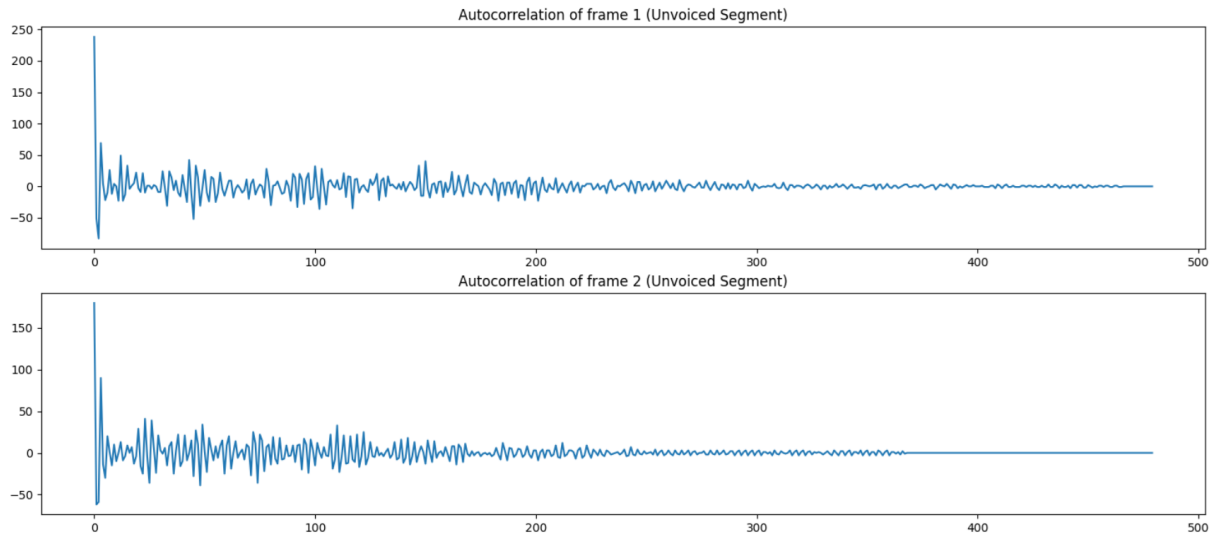


Figure 2.10: Autocorrelation Function applied at 2 Fully Unvoiced Frames

Figure 2.9 illustrates the autocorrelation of two frames that are characterised as 'Voiced', while Figure 2.10 illustrates the autocorrelation of two frames that are characterised as 'Unvoiced'. The difference is obvious. Due to the fact that voiced sounds are quasi-periodic, their autocorrelation function have a peak in each period, which is comparable with the autocorrelation value at zero lag (a property of the autocorrelation function is that it contains its highest value at lag 0), while in the aperiodic unvoiced sounds, their autocorrelation function doesn't have any strong peak. So, to eliminate some unvoiced segments, I will use the following criterion; if there is a lag inside a frame that its autocorrelation value exceeds the 30% of the autocorrelation of the lag 0, then it is characterized as voiced, and the concept of a pitch period exists. Otherwise, the frame is classified as unvoiced, and no further action will be taken (the frame doesn't have any strong peak). To detect the peaks, for each column of the autocorrelation array, I will use the 'find_peaks' function of the scipy library. If this function finds at least one peak, then it stores the index of the found peak in another array, otherwise, appends a zero.

After all this process, I managed to have an array, where it contains either 0, if there is no strong peak in the autocorrelation frame (automatically classified as unvoiced), or the index of sample in a frame for which the peak is found. The algorithm seems to work well till now, but I will use one more constraint to eliminate even more voiced samples. I will assume that if I find a peak before the 30th sample of the frame, I will discard it. Its fundamental frequency will

be $16000 / 30 = 533.33$ Hz, which is above the upper limit in human speech, which is 400Hz (especially in men, this limit is 200 Hz, but I don't want to impose any additional restrictions on my model). So, I will consider these peaks as outliers and I will remove them. Finally, in this array, I do not have the fundamental frequency, but the fundamental period. To find the fundamental frequency, all I need to do is to invert each fundamental period in the array. To avoid division by 0, I will only invert the indices with values greater than 30.

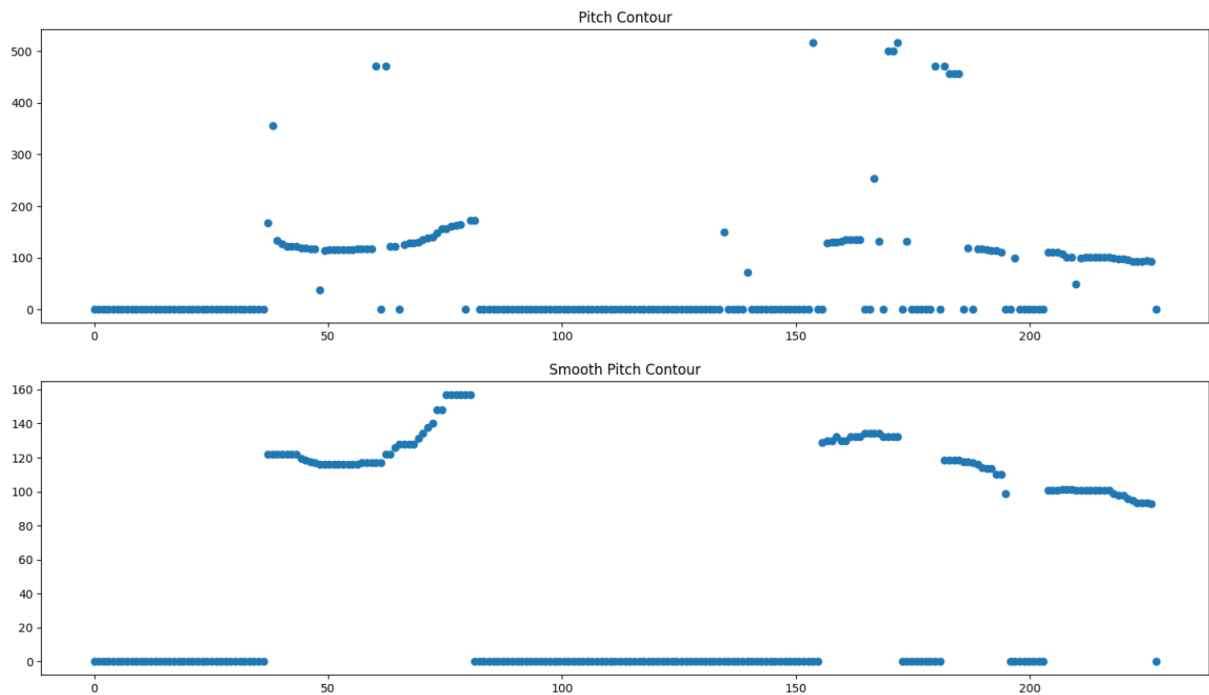


Figure 2.11: Pitch Contours Before and After the Smoothing Filter

The pitch contour obtained from the process that is described above is shown in the upper plot of Figure 2.11. We can see that the results are accurate and acceptable, despite the high variance among the data points. But by applying a median filter with a kernel size of 11, we obtain a smoother version of pitch contour.

2.4 Linear Predictive Analysis of Two Speech Segments

As a final part of this part of the assignment, I have to implement Linear Predictive Analysis (or Linear Predictive Coding, LPC shortly), in one 30msec voiced segment and one 30msec

unvoiced segment of the speech signal. As I said before, 30msec interval corresponds at 480 samples, at 16kHz sampling rate. So, I chose randomly 480 voiced and 480 unvoiced samples for this task.

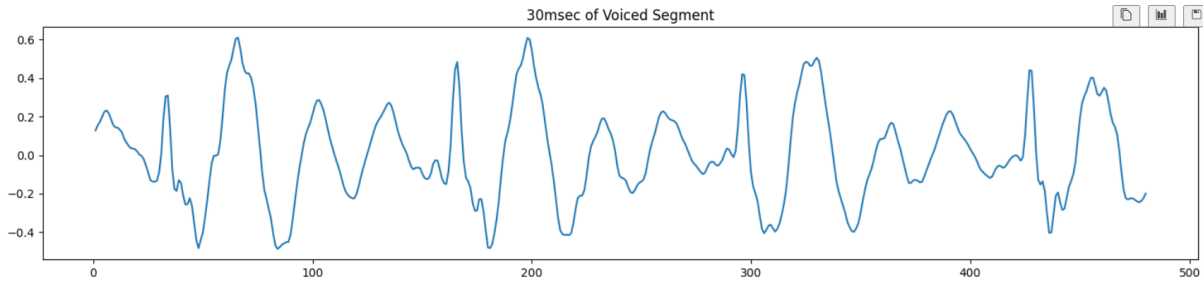


Figure 2.12: 30msec interval of a Voiced Segment

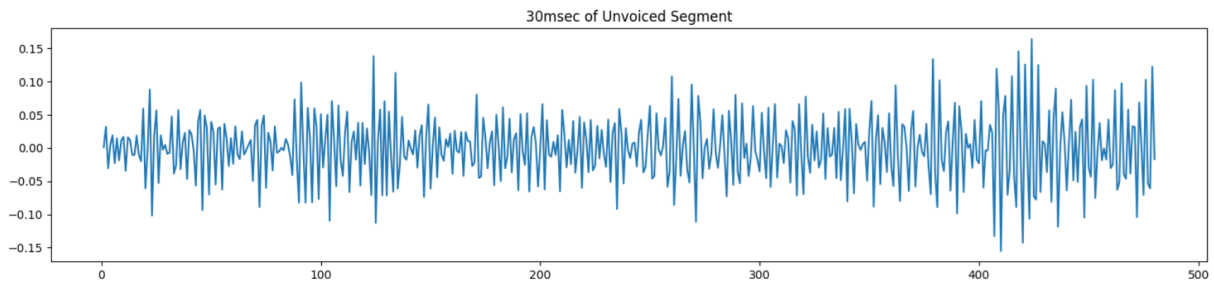


Figure 2.13: 30msec interval of a Unvoiced Segment

Figure 2.12 illustrates the 480 samples of a voiced segment (specifically, samples 10.120-10.600 of the cut name signal), while Figure 2.13 shows 480 samples of an unvoiced segment (samples 520-1.000 of the cut name signal). We can also see the lack of periodicity in the unvoiced segment, and the, almost periodic, voiced segment. We have already discussed many times about this property of voiced and unvoiced segments.

There are many methods to acquire the LPC coefficients. Some of them are the Autocorrelation Method, the Covariance Method and the Lattice Method. I have used the Autocorrelation Method in my project. In this method, I have to solve a linear system of p equations with p unknowns, which is written as $Ra = r$ in matrix form. To create the matrix R (which is named the autocorrelation array), and the target vector r , I call the 'create_autocorrelation_matrices' function. Then, all I have to do, is to solve the linear system (e.g. to inverse the R matrix) and multiply it with the target vector r . The inversion is really simple in python. There is the

`linalg.inv` function in `numpy` library that performs it. With this procedure, I have managed to acquire the vector a , by solving the simple $a = R^{-1}r$ linear system

To evaluate my results, I will use the Mean Squared Error (MSE), which is computed in the custom-made function 'compute_mse'. I will find the LPC coefficients 3 times, for $p=8$, $p=12$ and $p=16$, and I will compare the results (usually p is way much smaller than voiced and unvoiced segments, otherwise, the LPC analysis is meaningless).

```
MSE for the Voiced Segment and p=8: 0.3177743424373709, with Gain: 0.5637147704623064
MSE for the Voiced Segment and p=12: 0.2865009423549516, with Gain: 0.5352578279249651
MSE for the Voiced Segment and p=16: 0.27656016779273784, with Gain: 0.525889881812474
```

Figure 2.14: Mean Squared Error for Voiced Segment

```
MSE for the Unvoiced Segment and p=8: 0.17537745884262246, with Gain: 0.41878091986457844
MSE for the Unvoiced Segment and p=12: 0.1506695991232494, with Gain: 0.38816182079546335
MSE for the Unvoiced Segment and p=16: 0.1335091678266216, with Gain: 0.36538906363850243
```

Figure 2.15: Mean Squared Error for Unvoiced Segment

The results are depicted in Figures 2.14 and 2.15. All prediction errors are quite small, which mean that the LPC analysis is quite accurate. Also, we can notice that as the order p of the prediction increases, the MSE slightly decreases. This decrease makes sense, since having more samples before the prediction implies that the remaining samples will be predicted more accurately. But the purpose of LPC analysis, is to predict the signal accurately, with the lowest possible prediction order p . Otherwise, it's obvious that as p tends to the frame length, the MSE tends to 0. Thus, if someone wants to have a meaningful LPC analysis, he needs to find the proper order p for which he will have an acceptable error. This is not such a complicated task, but it is outside the assignment's requirements.

Lastly, I will plot the all-pole transfer function $H(z)$ of the voiced and the unvoiced segments for $p=8$, $p=12$ and $p=16$. The formula for the transfer function is

$$H(z) = \frac{G}{1 - \sum_{k=1}^p a_k z^{-k}}$$

where a_k are the LPC coefficients, p is the order of the prediction and G is the gain factor

(printed above along with the MSE). To compute G , I used the square root of the MSE.

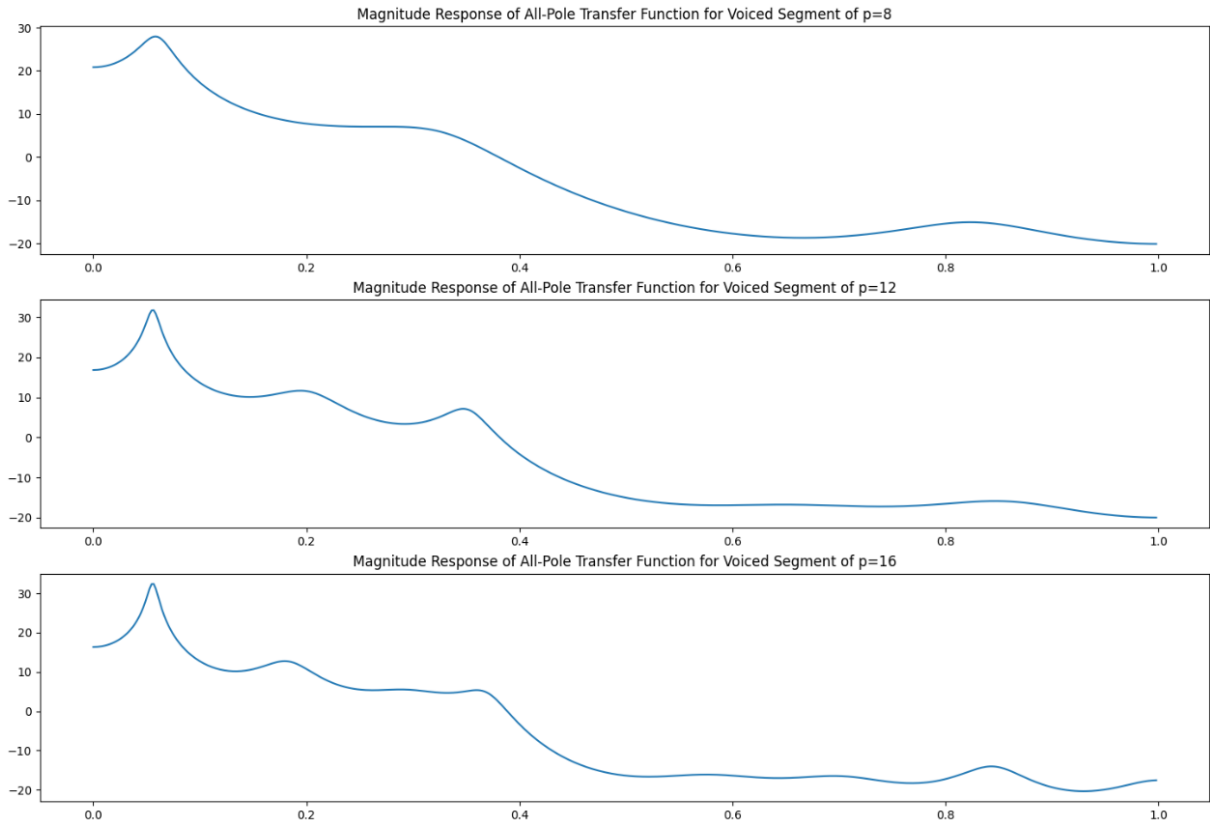


Figure 2.16: All-Pole Transfer Function for the Voiced Segment

Figure 2.16 illustrates the transfer function for voiced segments. It is clear that as the order p decreases, the curve becomes smoother and it is more difficult to distinguish the formants, which result in less accurate results. Figure 2.17 illustrates the transfer function for unvoiced segments. The main difference in the plots lies in the order of the formants. In voiced segments, the first formant is the one which dominates more (the one with the highest magnitude), the next one has the second highest magnitude and so on. For the unvoiced segments, the order of the formants is the opposite (the most dominant formant is the last one). The effect of the order p in the resolution of the formants can be seen in that plot too.

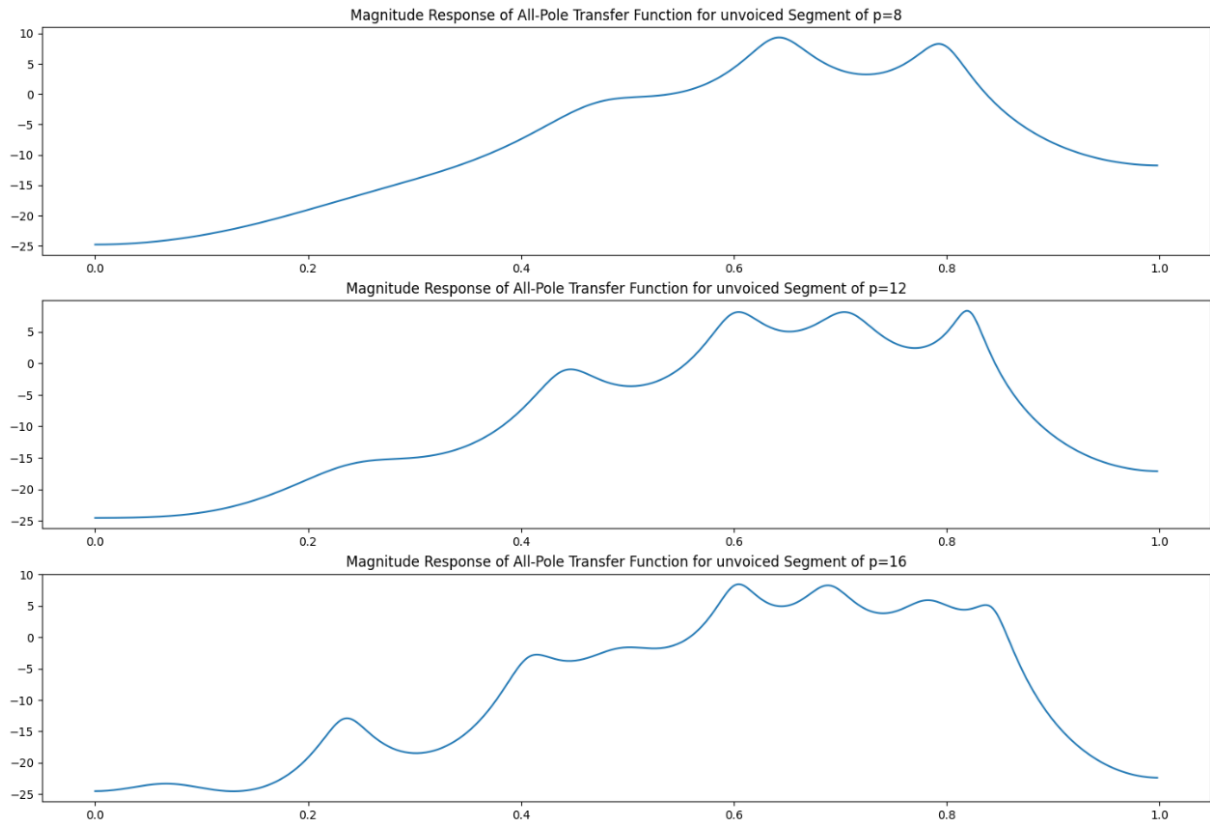


Figure 2.17: All-Pole Transfer Function for the Unvoiced Segment

3 Part B: A Simple Vowel Synthesizer

The task in this part is to create a simple vowel synthesizer for 6 different vowels. This synthesizer will be based on the assumption that natural speech (specifically voiced speech) is the output of an LTI system, which takes as input an impulse train $p[n]$ that is created at the vocal chords, it passes $p[n]$ through the vocal tract (we assume vocal tract as a combination of LTI filters), with a total impulse response $h_v[n]$, and creates an output signal named $s[n]$. So, all I have to do in this question, is to model these 'filters' $h_v[n]$ digitally, and pass through them an impulse train.

3.1 Human Vocal System

Let's delve into the vocal tract filters. Voiced sounds are created by the vibration of the vocal chords, and they are passing through some filters before exiting from the vocal tract. The first filter that a voiced sound enters into is named *Glottal Pulse*. The formula that describes this filter in time domain is

$$g[n] = \begin{cases} 0.5 \left[1 + \cos \left(\frac{(n+1)\pi}{N_1} \right) \right] & 0 \leq n \leq N_1 - 1 \\ \cos \left(0.5 \frac{(n+1-N_1)\pi}{N_2} \right) & N_1 \leq n \leq N_1 + N_2 - 1 \\ 0 & \text{otherwise} \end{cases}$$

where $N_1 = 25$ and $N_2 = 10$. In frequency domain, this filter is a *maximum phase filter*, e.g. all of its zeros are located outside the unit circle. The signal exits from this filter as a 'pulse', and this pulse enters the next filter, which is named as the *Vocal Tract Filter*. Its representation in z-domain is:

$$V(z) = \frac{1}{\prod_{k=1}^K (1 - 2e^{-2\pi\sigma_k T} \cos(2\pi F_k T) z^{-1} + e^{-4\pi\sigma_k T} z^{-2})}$$

where K is the number of formants (I will use 3 formants, so $K = 3$), $2\sigma_k$ is the bandwidth of each formant. (for the first formant, the bandwidth is 60Hz, for the second one is 100Hz and for the third one is 120Hz), T is the sampling period, which is set at 10^{-4} and F_k , are the formant frequencies in Hz. Each vowel has its own F_K frequencies, which are depicted in the table below

	F1	F2	F3
/AO/	570	840	2410
/IY/	270	2290	3010
/UH/	440	1020	2240
/EH/	530	1840	2480
/AH/	520	1190	2390
/IH/	390	1990	2550

Table 3.1: Formant Frequencies for each Vowel (in Hz)

And finally, the last filter that the signal enters, is named *Radiation Load*, and models the pressure at the lips. Its formula in time domain is really simple:

$$r[n] = \delta[n] - \gamma\delta[n - 1]$$

where γ is a constant set at 0.96, and $\delta[n]$ is the unit impulse.

Since I have assumed an LTI system, it is obvious that the output of the whole system will be:

$$s[n] = A(p[n] * g[n] * v[n] * r[n])$$

where A is the gain factor (constant, set at 5.000). So, it is obvious that if I manage to model the impulse train and all these filters, I will acquire my $s[n]$ signal, which will be the synthetic vowel.

Fortunately, the only thing that distinguishes one vowel from another is the formant frequencies, and the only function that depends on them is the vocal tract function.

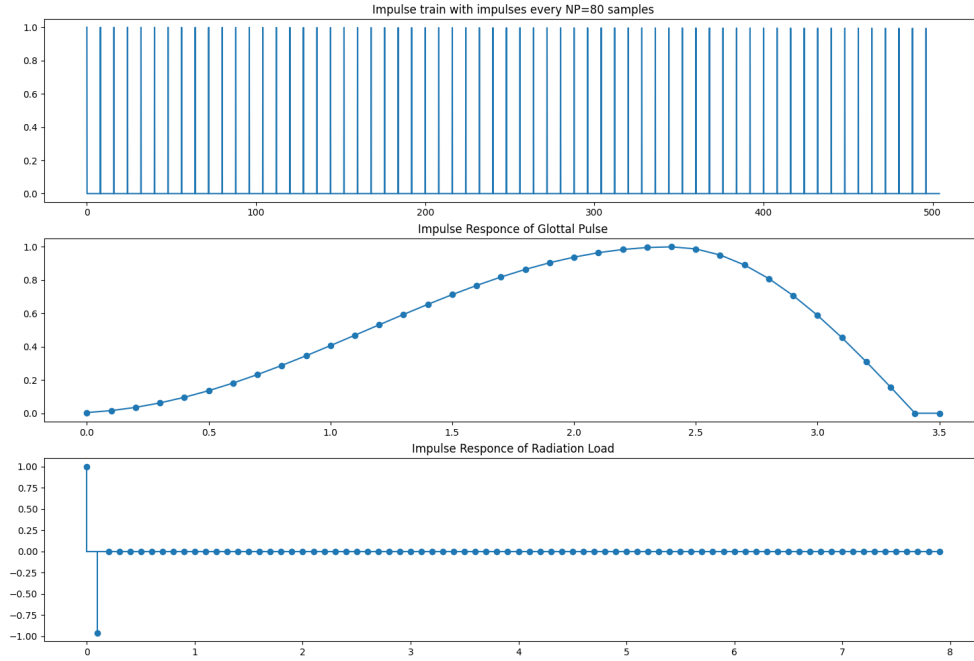


Figure 3.1: Filters of the Vocal System in Time Domain (in msec)

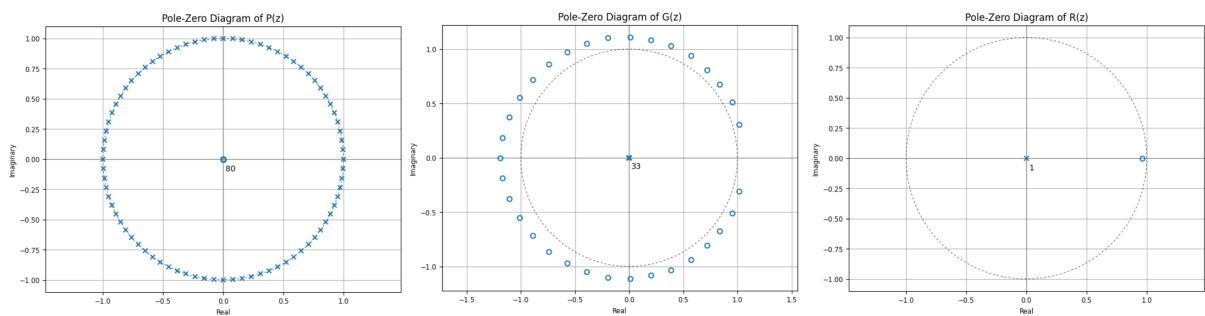


Figure 3.2: Pole-Zero Diagrams of the Vocal System Transfer Functions

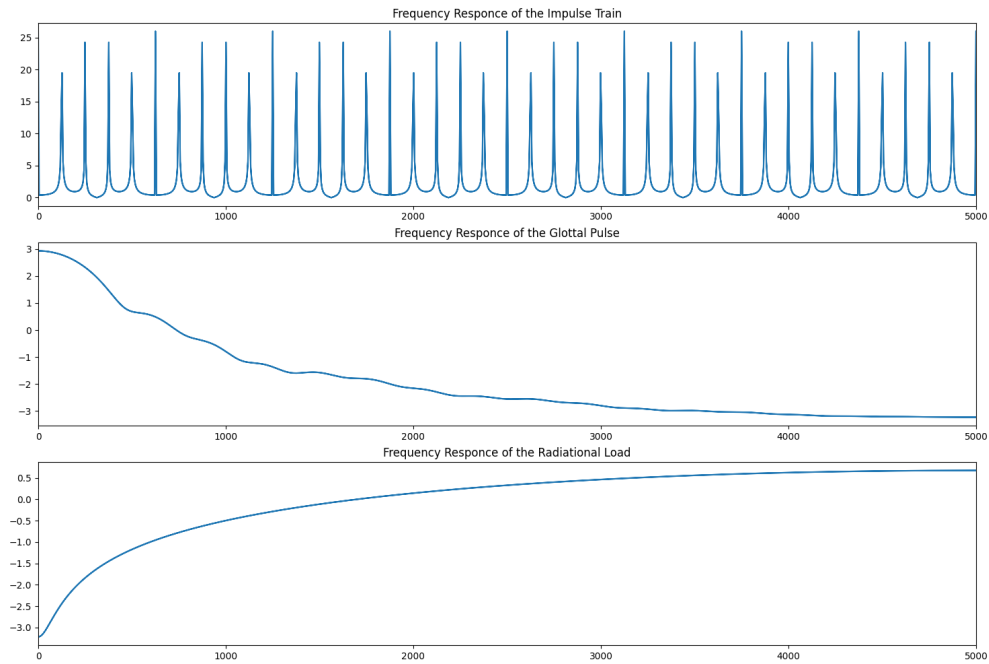


Figure 3.3: Filters of the Vocal System in Frequency Domain

Figures 3.1, 3.2 and 3.3 illustrate the filters of the vocal system that do not change when a vowel changes (e.g. they are independent of the formant frequencies). Figure 3.1 illustrates the impulse responses of these filters, while Figure 3.2 illustrates the Pole-Zero Diagrams of the transfer functions of these filters. Finally, Figure 3.3 shows the frequency response of these filters, which is obtained by using a 2048-point FFT in the impulse responses.

Now I will move on to the vocal tract filter, which will have different impulse and frequency response for each vowel. In the following figures (Figure 3.4 and Figure 3.5), this filter will be depicted for each vowel in time and frequency domain correspondingly.

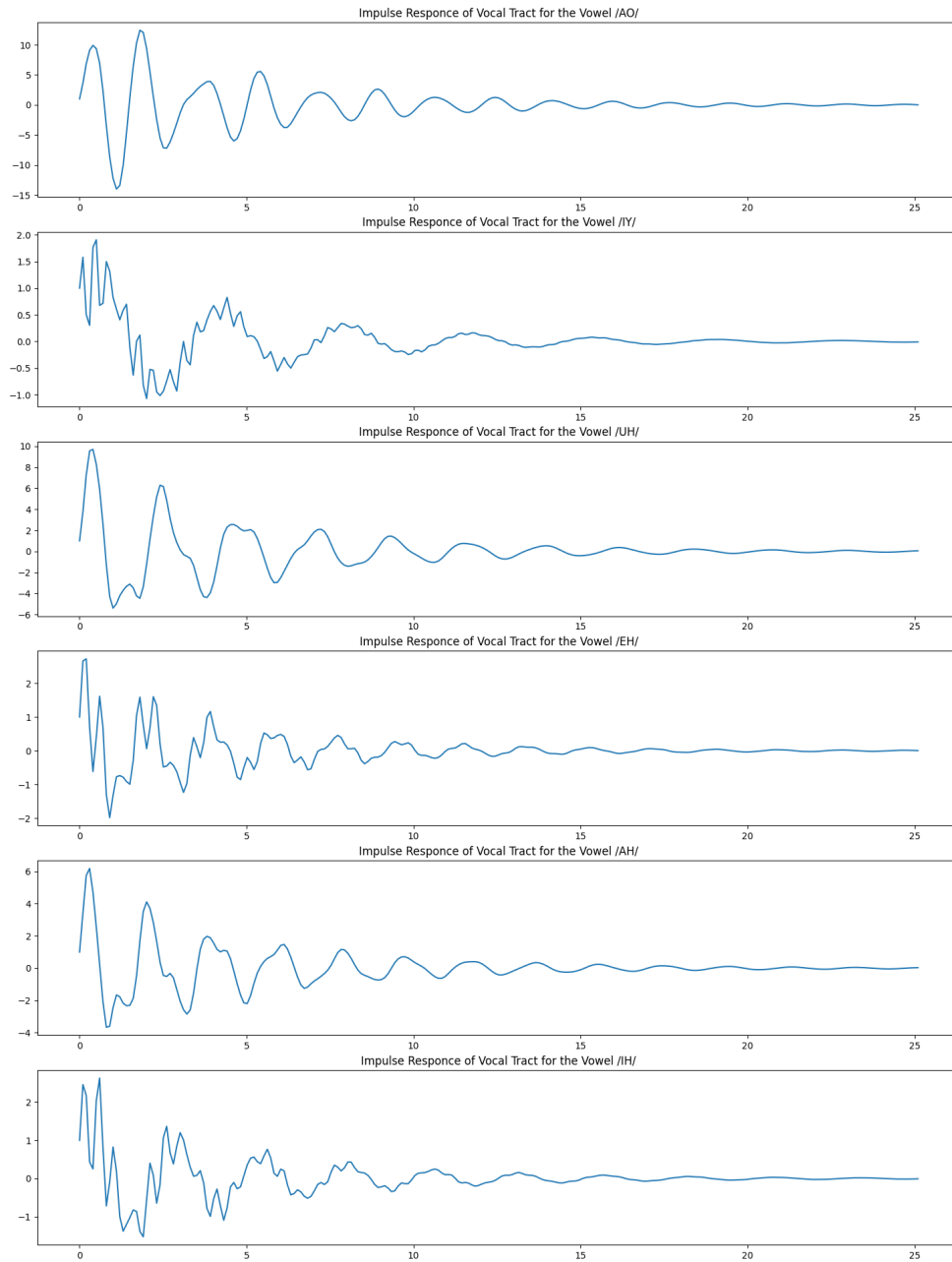


Figure 3.4: Impulse Response of the Vocal Tract for Each Vowel

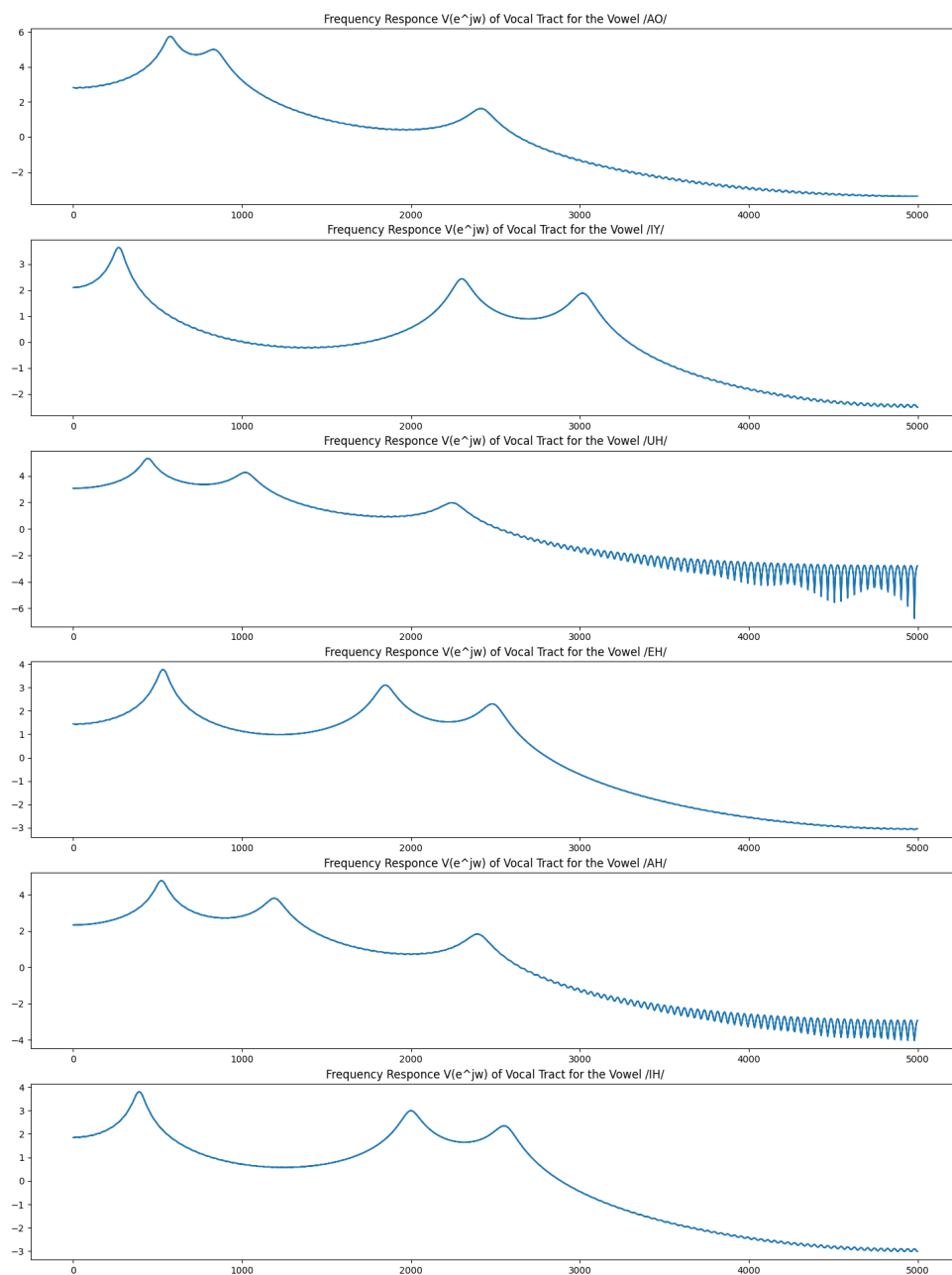


Figure 3.5: Frequency Response of the Vocal Tract for Each Vowel

The effect of each vowel in the vocal tract is clear. In the frequency response, we can easily distinguish the formants (each peak corresponds to one formant). From the decaying form of the formants, we can verify that a vowel (or a voiced sound, if we want to be more explicit) passes through the tract. In comparison to the glottal pulse, this filter is a *minimum phase filter*, which means that it has all of its zeros inside the unit circle (and since the numerator is equal to 1, all of its zeros are located at the origin). We can also see the Pole-Zero Diagrams of the transfer function of the vocal tract for each vowel separately in the Figure 3.6.

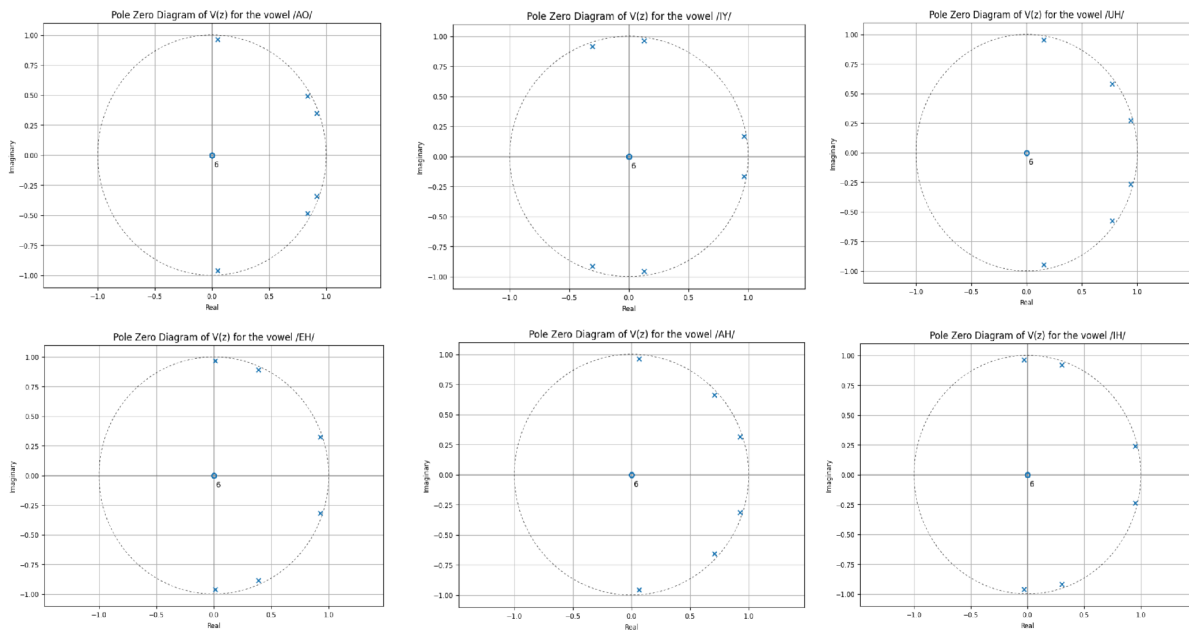


Figure 3.6: Frequency Response of the Vocal Tract for Each Vowel

3.2 Synthesize Vowels

Now that I have modelled all filters of the human vocal system, it's time to convolve the impulse train, which contains a sample with an amplitude close to 1 every 80 samples, with each vocal system filter in the correct order.

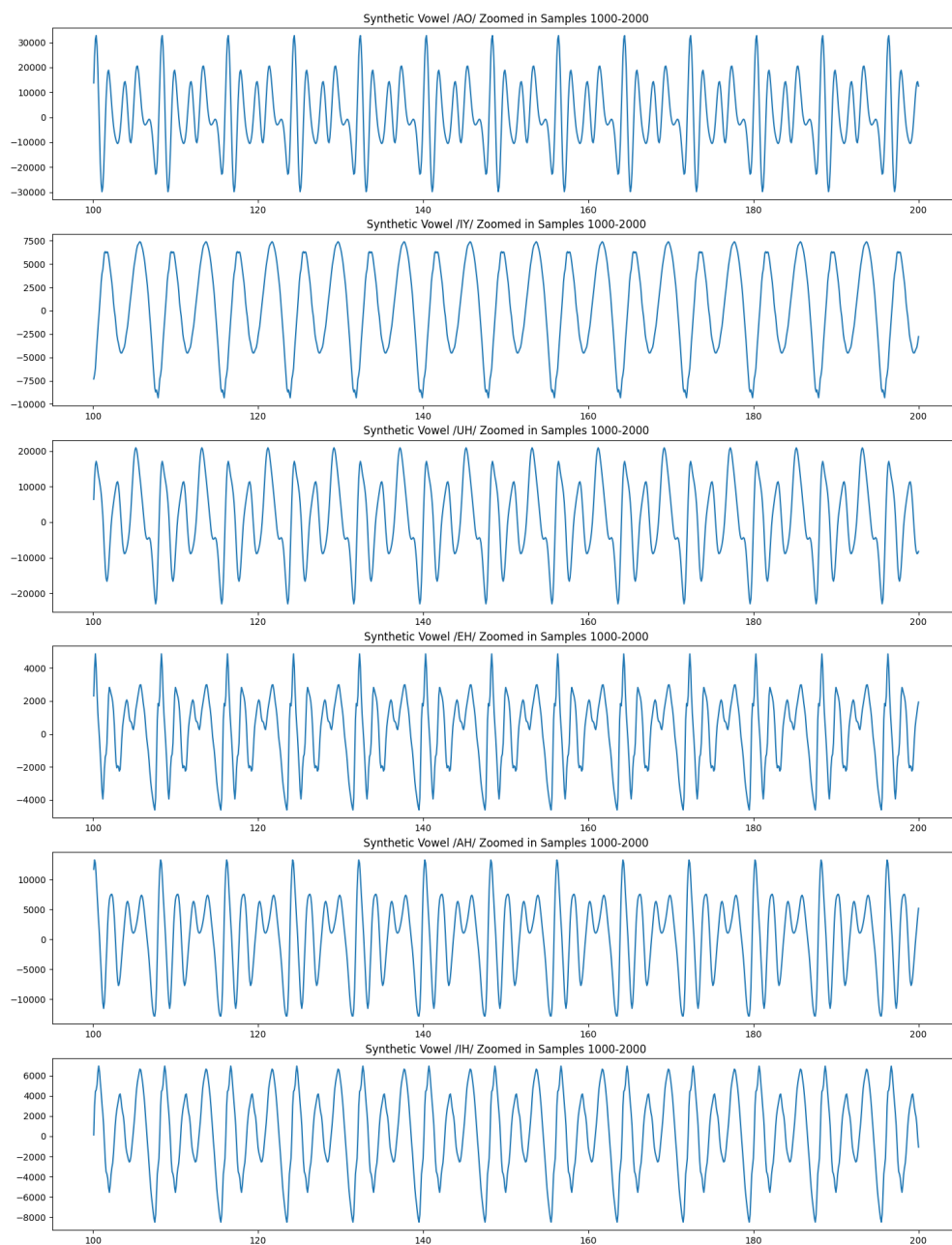


Figure 3.7: Synthetic Vowels in Time Domain

Figure 3.7 illustrates a part of the speech signal $s[n]$, for each synthetic vowel, which is found by using the formula described in the previous section. I chose to plot 1000 samples randomly, between 1000 and 2000, but due to the periodicity, you can plot other intervals too, and obtain the same results. In comparison with natural speech, these waveforms are fully periodic. In the next figures (Figure 3.8 and Figure 3.9) you can see the Fourier Transform of each vowel, and the Pole-Zero Diagrams of each vowel. For the creation of the Pole-Zero Diagram, I had to multiply the frequency response of each filter (since I have an LTI system, the transfer function of the whole system will be the multiplication of the transfer function of each filter), and then, I had to find the zeros and the poles of the new, combined, transfer function.

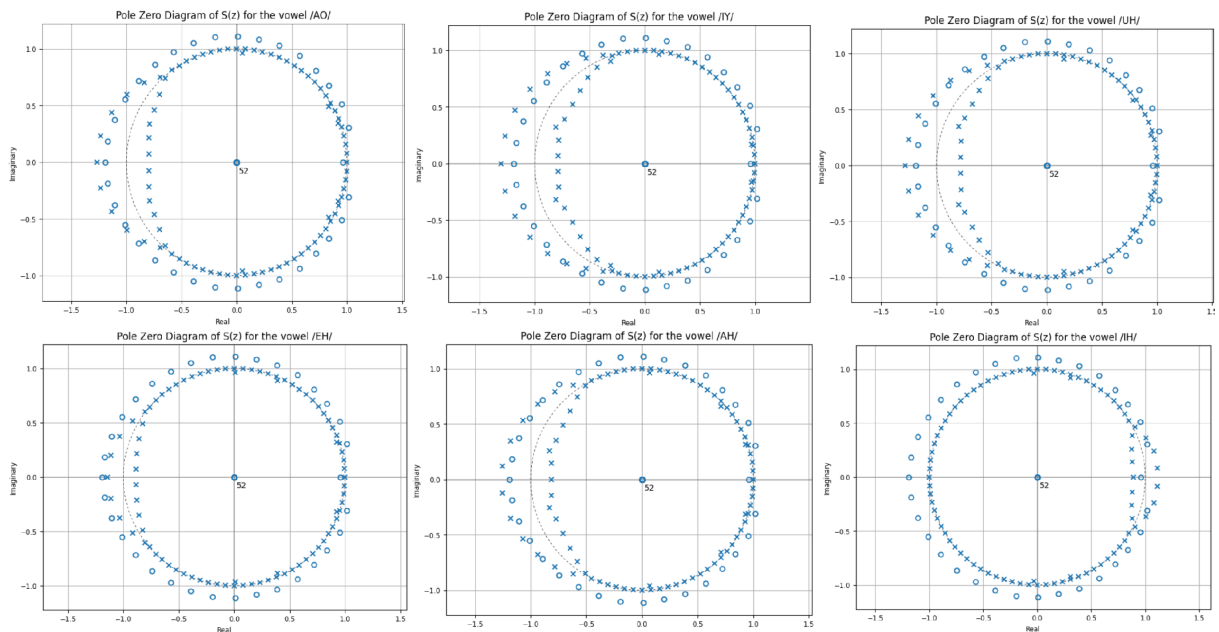


Figure 3.8: Pole Zero Diagrams of the Combined Transfer Function for Eaxh Vowel

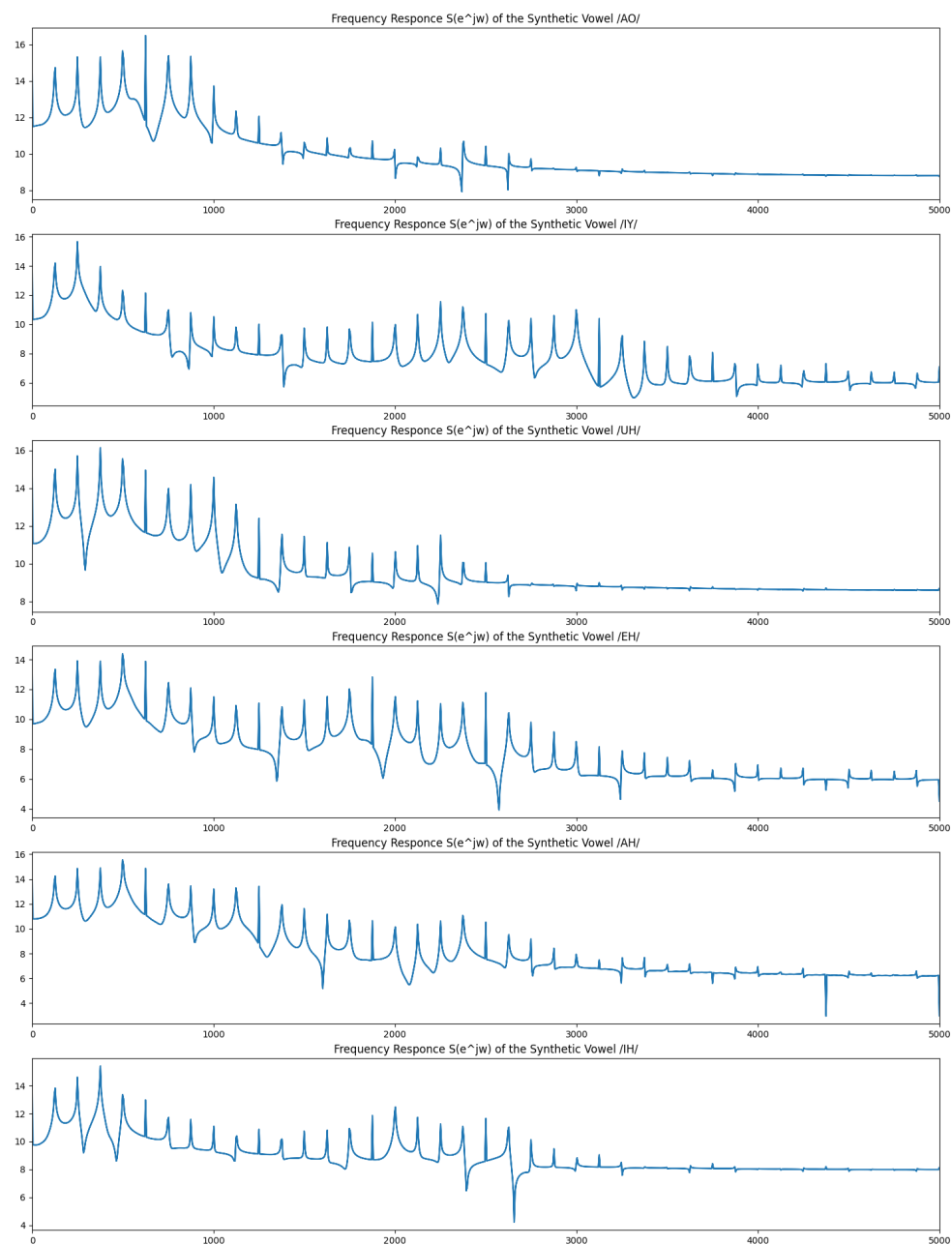


Figure 3.9: Synthetic Vowels in Frequency Domain

Now that everything is ready, I will concatenate each vowel to create a single signal. This signal is named 'synthetic_vowels.wav' and it is located in the file named 'wav_files'. By hearing this audio file, you can verify that each vowel is synthesized successfully.

3.3 Create the Necessary Spectrograms

And as a final task in this assignment, I have to create two spectrograms. The first one will show the audio file with the synthesized vowels, and the second one, the same vowels recorded by me and sampled at 16kHz. For each analysis, I will use 100 frames per second with 75% overlap. The results can be seen in Figure 3.10 below.

The upper plot shows the spectrogram of the synthetic vowels, while the lower shows the spectrogram of the natural vowels. We can observe that the upper spectrogram is the ideal version of the lower one, but due to physical limitations, the human vocal tract cannot produce a perfectly periodic waveform. However, the lower spectrogram provides remarkable results that can be successfully processed and analyzed, allowing for the creation of highly accurate models.

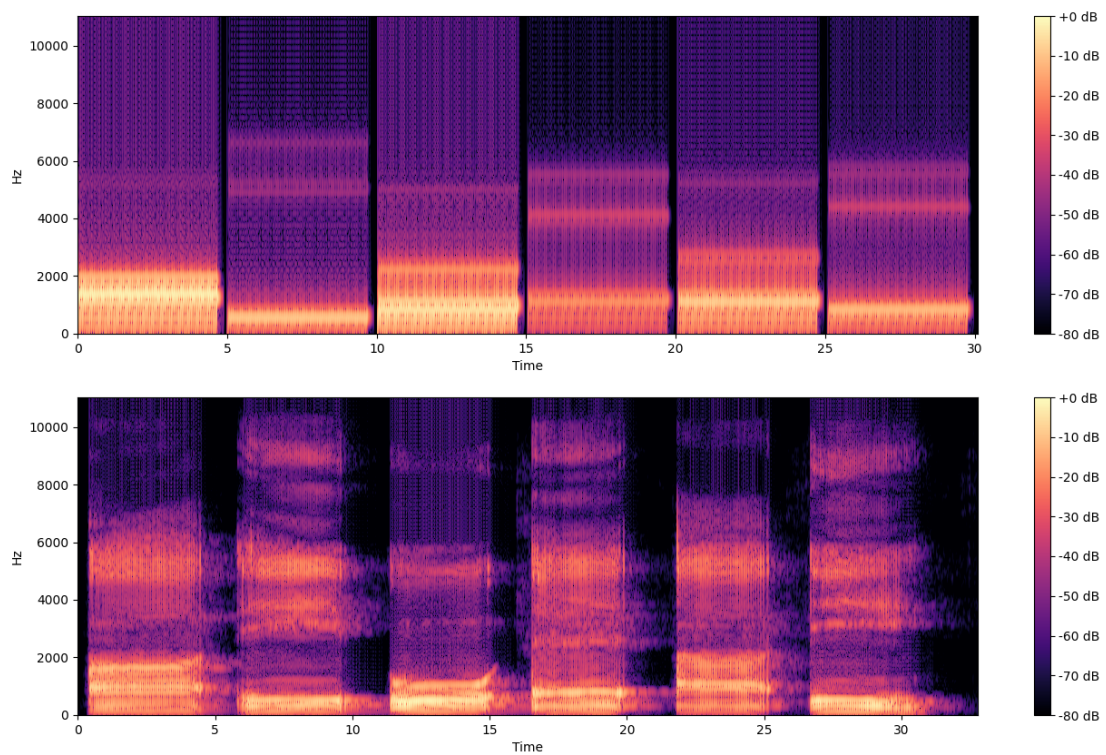


Figure 3.10: Spectrograms of both Synthetic and Natural Vowels